OpenZeppelin | security

OpenZeppelin Stack for Zama: Smart Accounts Audit



January 7, 2026

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Summary 5

Components 5

Security Model and Trust Assumptions 6

Critical Severity 8

C-01 Inner userOp Signature Is Not Verified in _validateEnableMode 8

Low Severity 9

L-01 Incomplete Docstrings 9

L-02 Non-payable Executor Entry Point Cannot Forward ETH to the Account 9

L-03 Self-signer Contract Accounts Enable ERC-1271 Recursion and Out-of-Gas in Validation 10

L-04 Unbounded executeFromExecutor Return Data Enables Gas/Memory DoS 11

L-05 Empty Return Value in ERC7579DelayedExecutor._scheduleAt 12

L-06 Install Path Does Not Honor minSetback 13

Notes & Additional Information 14

N-01 ABI-level Malleability in operationId Yields Different IDs for the Same Effect 14

N-02 Bounds Check in Multisig Weighted Init Decoder Causes Panic Instead of Custom Error 15

N-03 Inaccurate or Outdated Documentation 16

N-04 Redundant Return Statement 16

N-05 _removeSigners Deletes Weight Data Before Verifying Signer Existence 18

N-06 Missing Bounds Validation for Array Data Length in Decoding Functions 18

Conclusion 20

Appendix 22

Issue Classification 22

Summary

Type	Library
Timeline	From 2025-12-10 To 2025-12-19
Languages	Solidity

Total Issues	13 (10 resolved, 1 partially resolved)
Critical Severity Issues	1 (1 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	6 (4 resolved, 1 partially resolved)
Notes & Additional Information	6 (5 resolved)

Scope

OpenZeppelin audited the open-accounts-framework repository at commit 6e14ef8.

In scope were the following files:

```
packages
└── contracts
    └── src
        └── modules
            ├── ERC7579Multisig.sol
            ├── ERC7579MultisigWeighted.sol
            ├── executors
            │   ├── ERC7579DelayedExecutor.sol
            │   └── ERC7579Executor.sol
            └── validators
                └── ERC7579Validator.sol
```

System Overview

Summary

The files in scope represent the key building blocks for EIP-7579 (Minimal Modular Smart Accounts): Validators, Executors, and multisignature validation aligned with EIP-7913 (Signature Verifiers). Collectively, these contracts provide:

- **execution capabilities** for modular smart accounts via ERC-7579 executor modules, including a minimal executor and an enhanced executor supporting time-delayed scheduling with expiration.
- **validation capabilities** for ERC-7579 accounts, including integration with EIP-4337 (Account Abstraction Using Alt Mempool) `UserOperation` validation, and EIP-1271 signature validation through a shared validator base.
- **multisignature primitives** (signer management, thresholds, and signature checking) usable by validators or privileged flows, including both count-based and weighted threshold schemes.

ERC-7579 defines four module types—Validator, Executor, Fallback Handler, and Hook, of which this scope covers Validators and Executors. The overall architecture follows ERC-7579's modular design principles: accounts install and rely on external modules for specific behaviors while preserving interoperability across implementations. As a result, the effective security properties depend not only on each module's correctness but also on the account's module permissioning, the installation/uninstallation lifecycle, and the authorization policies implemented in derived modules.

Components

The following system components were identified:

- `ERC7579Executor.sol`: A minimal abstract executor with a public `execute(...)` entry point that delegates authorization to `_validateExecution` and dispatches to the account via `executeFromExecutor(...)`.
- `ERC7579DelayedExecutor.sol`: It extends the executor with per-account delay/expiration configuration, operation scheduling/cancellation, and on-chain enforcement of an operation lifecycle (`Scheduled → Ready → Executed/Expired/Canceled`).

- `ERC7579Validator.sol`: An abstract validator module that integrates ERC-4337 `validateUserOp` and ERC-1271-style signature semantics via `isValidSignatureWithSender`, delegating cryptographic checks to `_rawERC7579Validation`.
- `ERC7579Multisig.sol`: An ERC-7913 signer-set + threshold management contract. It uses helpers to validate a set of signer signatures against a threshold.
- `ERC7579MultisigWeighted.sol`: It extends multisig with per-signer weights, enforcing thresholds against total weight and supporting batched weight updates.

Security Model and Trust Assumptions

Throughout the in-scope codebase, the following security assumptions were identified:

- **Account enforcement (external trust assumption):** The account correctly restricts `executeFromExecutor` and signature validation to installed modules.
- **Authorization hooks (implementation assumption):** Derived executors must correctly implement `_validateExecution`, and the delayed executor must correctly implement `_validateSchedule/_validateCancel`.
- **Time-delayed execution (operational requirement):** Monitoring/cancellation must exist to benefit from the delay window. The intended operations must be executed during the Ready window otherwise they will expire.
- **Persistence and lifecycle:** The delayed executor does not clear scheduled operations on uninstall. As such, systems must account for schedule persistence across reinstall/upgrade flows.
- **Multisig input format and config (assumption/requirement):** Signer lists must not double-count signers (via uniqueness/canonical ordering or downstream checks), and weighted thresholds must be configured consistently with the weight scale. Signer-set size should remain within practical gas limits.
- **ERC-7913 verifiers must use a canonical key encoding**: For a given verifier, different key-byte strings must not represent the same signer (i.e., the verifier must not ignore/normalize parts of the key). Otherwise, one signer could be registered multiple times under different encodings and be counted more than once.

- The initialization data passed as input during the installation of the ERC-7579 delayed executor module is assumed to be encoded using packed encoding (i.e., with `abi.encodePacked` and not with `abi.encode`).
- There are no hardcoded admin or owner roles in the base contracts. All privileged actions flow through the account or are gated by abstract validation functions that must be implemented by derived modules.

Critical Severity

C-01 Inner `userOp` Signature Is Not Verified in `_validateEnableMode`

The enable-mode extension is intended to install modules and then proceed with the normal user operation validation in the same request. The `_validateUserOp` documentation describes steps "*1-4*", ending with "*Continues with normal user operation validation*" after installation.

`_validateUserOp` is the function that installs the module and executes the user operation atomically in a single call in enable-mode. It first checks the EIP-712 signature of the module in a call to `_validateEnableMode`. Then, it installs the module if verification succeeds and returns `SIG_VALIDATION_SUCCESS` - the success path. If verification fails, `_validateEnableMode` returns `SIG_VALIDATION_FAILED` and does not install the module - the fail path. In the fail path, `_validateUserOp` enters the `if`-condition, then installs the module if not already installed and finally verifies the signature of the user operation through a call to `_validateUserOp`.

However, in the success path (i.e., when module installation by `_validateEnableMode` succeeds), the function returns success immediately, meaning the **inner user operation signature is never checked**. Thus, an attacker with a valid installation signature can install a validator and execute arbitrary calls in the same user operation without presenting a valid user operation signature.

Consider validating the inner user operation signature after a successful module installation in `_validateEnableMode` and updating the documentation for `_validateUserOp` accordingly.

Update: *Resolved in pull request #23. The team stated:*

> *The `validateEnableMode` function now checks the userOp signature.*

Low Severity

L-01 Incomplete Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- In `ERC7579Multisig.sol`, within the `ERC7913SignerAdded` event, the `account` and `signer` parameters are not documented.
- In `ERC7579MultisigWeighted.sol`, within the `ERC7579MultisigWeightChanged` event, the `account`, `signer`, and `weight` parameters are not documented.
- In `ERC7579DelayedExecutor.sol`, within the `ERC7579ExecutorOperationScheduled` event, the `account`, `operationId`, `salt`, `mode`, `executionCalldata`, and `schedule` parameters are not documented.
- In `ERC7579Validator.sol`, within the `isValidSignatureWithSender` function, the `None`, `hash`, and `signature` parameters, along with some return values, are not documented.

Note that the above list is not exhaustive.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the Ethereum Natural Specification Format (NatSpec).

Update: *Acknowledged, not resolved. The team stated:*

> *The `@param` and `@return` tags are purposely ignored across the contracts DocStrings. The team considers that they often provide repetitive information about the arguments or return values.*

L-02 Non-`payable` Executor Entry Point Cannot Forward ETH to the Account

`ERC7579Executor` is the base entry point for ERC-7579 execution modules. The `execute` function is declared without the `payable` modifier and validates/dispatches to the internal `_execute`, which performs an external call to the account's `IERC7579Execution.executeFromExecutor`. The call is made without `{value: ...}`,

and therefore forwards zero wei. See `execute`, `_execute`, and the ABI for `executeFromExecutor`, which is `payable` and returns the subcall return data.

As a consequence, it is not possible to top up the account when triggering an execution through the executor. Any operation that requires sending ETH from the account (e.g., a subcall with non-zero `value`) will revert if the account has an insufficient balance, despite the account interface being designed to accept value from executors. This causes value-carrying executions to fail until the account is pre-funded via a separate transaction or the executor supports value forwarding.

Consider making `execute` `payable` and forwarding `msg.value` to the account (e.g., `IERC7579Execution(account).executeFromExecutor{value: msg.value}(...)`). In addition, consider documenting the expected funding flow and adding tests to ensure that value-carrying subcalls succeed when `msg.value` is provided.

Update: *Resolved in pull request #25. The team stated:*

> *The `execute` function is now payable and the internal `_execute` forwards `msg.value` accordingly.*

L-03 Self-signer Contract Accounts Enable ERC-1271 Recursion and Out-of-Gas in Validation

The multisig validator validates signatures by delegating the list of ERC-7913 signers to `SignatureChecker`. After checking authorization, it directly calls `hash.areValidSignaturesNow(signers, signatures)`. Signers are accepted as any `bytes` with length >= 20, without prohibiting the account's own address as a 20-byte signer (`_addSigners`). When the signer is the account itself and a contract, the `SignatureChecker` path falls back to ERC-1271 and calls `account.isValidSignature(...)`, which is implemented via the installed validator's `isValidSignatureWithSender` entry point.

This creates a loop: the account's ERC-1271 flows into the validator, which decodes the multisig payload and again reaches `areValidSignaturesNow`. According to EIP-7579 when ERC-7579 account implements ERC-1271 via installed validators, it is doing a forwarding step: `account.isValidSignature(hash, signature)` → `validator.isValidSignatureWithSender(msg.sender, hash, signature)` → return ERC-1271 magic value. In the implementation, `isValidSignatureWithSender` calls `_rawERC7579Validation` which, if it returns `true`, calls `IERC1271.isValidSignature`, which triggers the following loop:

`_validateSignatures` → `areValidSignaturesNow` → `isValidSignatureNow` → `isValidERC1271SignatureNow` → `IERC1271.isValidSignature` → enter loop → `isValidSignatureWithSender` → `IERC1271.isValidSignature` → `isValidSignatureWithSender` → `IERC1271.isValidSignature` → ...

With a crafted nested payload that includes the same 20-byte self-signer, the process continues recursively until call depth or gas is exhausted. The module does not reject self-signers nor implement recursion limits, so a misconfigured 1-of-1 or 1-of-N policy can experience deterministic out-of-gas during signature validation.

Consider forbidding the registration of 20-byte signers whose leading 20 bytes equal the target account (e.g., in `_addSigners`), or ensuring that the `SignatureChecker` path does not invoke ERC-1271 when the verifier address equals the account. Alternatively, consider adding an explicit recursion guard or context flag in validation to short-circuit self-referential checks. At the minimum, consider documenting the observed behavior.

Update: *Partially Resolved in pull request #32. The team stated:*

> *Explicitly checking for the account's own address would provide false security, as the same recursive behavior can be triggered through:*
>
> *- Dead or invalid addresses (e.g., `address(0)`)*
>
> *- Custom ERC-7913 verifiers that call back to the account*
>
> *- Any contract implementing ERC-1271 by forwarding to this account*
>
> *Since there's no practical way to prevent all recursive or invalid scenarios, we've added clear documentation to `ERC7579Multisig._addSigners` warning developers about this responsibility.*
>
> *The documentation now explicitly states that integrators must validate that signers represent valid entities and avoid configurations that could compromise security or functionality.*

L-04 Unbounded `executeFromExecutor` Return Data Enables Gas/Memory DoS

The `_execute` function of the `ERC7579Executor` contract emits an event and immediately returns the `bytes[]` produced by the account's `executeFromExecutor`. Since the interface returns a dynamic array of bytes, the executor must allocate and copy the full return

buffer to memory before handing it to its own caller. See `_execute` and the ABI for `executeFromExecutor`.

An adversary-controlled target can return arbitrarily large data, causing high memory expansion and copy costs at the executor's caller. Deployments that expose `execute` to untrusted parties (e.g., relayers, public UIs) are vulnerable to transaction-level gas/memory DoS without affecting account state or funds.

Consider reducing the impact surface by limiting or truncating returned data (e.g., by capping the number of entries or bytes or returning a hash/length only). Consider adding an execution mode that suppresses return data, or enforcing access control on `execute` to prevent untrusted callers from triggering heavy-return operations. Documenting expected bounds can also help integrators size gas appropriately.

Update: *Resolved in pull request #33. The team stated:*

> *The return data is mandated by the ERC-7579 standard as part of the `IERC7579Module` interface, so any gas costs from large return data are inherent to the standard's design. Although it would be useful to limit or truncate the return data, it takes a single adversary-controlled target contract to return enough data to block the operation's execution.*
>
> *In the context of ERC7579Executor we added a note mentioning the issue in `_execute` . However, we think this is more relevant in the context of delayed operations where a single malicious target could block the entire operation's execution at will. Such case is also documented with a note in ERC7579DelayedExecutor.*

L-05 Empty Return Value in `ERC7579DelayedExecutor._scheduleAt`

In `ERC7579DelayedExecutor.sol`, the `_scheduleAt` function returns `id` and `schedule_`. However, the `schedule_` output is never computed. Instead, only `id` and `_schedules[id]` are computed.

Consider replacing `schedule_` with `_schedules[id]` in the return statement.

Update: *Resolved in pull request #31. The team stated:*

> *The `Schedule` struct was removed from the returned values of the `_scheduleAt` function. Developers can rely on `getSchedule` instead.*

L-06 Install Path Does Not Honor `minSetback`

In `ERC7579DelayedExecutor`, `onInstall` calls `_setDelay(account, initialDelay, 0)`, always passing `0` as the minimum setback. While this is correct for a fresh install (no existing delay config), it breaks the `minSetback()` invariant when a previous delay exists from an earlier installation and uninstall.

Scenario

- Module installed with `delay = 7 days`.
- `onUninstall` schedules `delay → 0` with `minimumSetback = minSetback() = 5 days`, so the effective delay remains 7 days until the transition completes.
- Before that transition takes effect, the module is reinstalled with `initialDelay = 4 days`.
- `onInstall` calls `_setDelay` with `minimumSetback = 0`, even though the old effective delay is still 7 days (pending reset to 0).
- The resulting setback is computed from the previous delay without applying `minSetback()`, allowing the new 4-day delay to become effective in 3 days.
- By contrast, a direct `setDelay(4 days)` from 7 days would enforce a setback of at least `minSetback() = 5 days`.

The uninstall → reinstall path provides a way to reduce the effective delay faster than the `minSetback()` protection.

Consider having `onInstall` detect whether a prior delay configuration exists and, in that case, call `_setDelay` with `minimumSetback = minSetback()` instead of `0`. Doing so would help ensure that reinstalling cannot bypass the minimum setback window.

Update: *Resolved in pull request #30. The team stated:*

> *The `onInstall` function now checks the `currentDelay` and calls `_setDelay` with `minSetback()` if there's a previous delay different from 0. This would enforce the minimum setback even if the module is uninstalled and then reinstalled.*

Notes & Additional Information

N-01 ABI-level Malleability in `operationId` Yields Different IDs for the Same Effect

The executor defines the identity (`id`) of an operation as `keccak256(abi.encode(account, salt, mode, executionCalldata))` in `hashOperation`. The `id` is then recomputed in `_scheduleAt`, `_execute`, and `_cancel`. However, two of the input parameters determining the `id`, namely `mode` and `executionCalldata`, are ABI-level malleable, meaning that they can be modified while still resulting in a valid `id` for the same operation in terms of final effect.

The `mode` parameter is of type `bytes32` and consists of two bytes that encode the call type and the execution type with the remaining 30 bytes being unused or reserved, as described in ERC-7579. By modifying the unused or reserved bytes, one may produce different `id`s for the same operation (same call type and execution type). Similarly, if the account's executor accepts semantically equivalent encodings of `executionCalldata`, two encodings that execute identically will map to different `operationId`s. For example, if decoding ignores unused trailing bytes then `executionCalldata = encodeBatch(calls)` and `executionCalldata = encodeBatch(calls) || 0xdeadbeef` will be semantically equivalent.

Off-chain tools commonly re-encode calldata canonically when canceling, which makes the `id`s diverge due to the noted malleability effects. This causes practical cancellation failures or "stuck" schedules: `_cancel` only allows `Scheduled`/`Ready` and reverts for `Unknown`, while `_execute` requires `Ready`. If the recomputed ID does not match the scheduled one, cancellations revert and later executions using the original bytes succeed. The issue does not bypass authorization but reduces the reliability of the delay window and monitoring.

Consider normalizing or constraining `mode` and `executionCalldata` before hashing (e.g., by enforcing reserved and unused bytes to zero and rejecting trailing data). Alternatively, consider adding a cancel-by-ID interface so that off-chain systems can use the exact ID emitted in the event `ERC7579ExecutorOperationScheduled`. Document the expected `mode` and `executionCalldata` canonical form and encourage clients to reuse the bytes from the schedule event when canceling.

Update: *Acknowledged, not resolved. The team stated:*

> *This issue is acknowledged, but does not pose a security risk and normalization of* `executionCalldata` *is impractical. On the one hand, the* `mode` *argument would prevent accounts from using reserved bytes for legitimate custom purposes as permitted by ERC-7579. On the other hand, normalizing* `executionCalldata` *would require recursively validating the internal* `callData` *and checking for trailing bytes as well. Trailing data is standard Solidity behavior when calling functions, so we consider the current behavior matches normal Solidity functionality.*
>
> *Malleability only affects operational convenience when off-chain tools re-encode parameters. Clients should look at the exact bytes from the* `ERC7579ExecutorOperationScheduled` *even when canceling operations, rather than re-encoding.*

N-02 Bounds Check in Multisig Weighted Init Decoder Causes Panic Instead of Custom Error

The `_decodeMultisigWeightedInitData` function first extracts the `weightsOffset` and immediately reads the weights array length from `initData[weightsOffset:weightsOffset + 0x20]`. The bounds validation for `weightsOffset` occurs afterwards, so an out-of-range offset triggers a low-level out-of-bounds panic instead of the intended `ERC7579MultisigInvalidInitData` error. This weakens error reporting and can make integration tests brittle.

Consider validating `weightsOffset` before any slice using it (e.g., check `weightsOffset > 0x3f` and `weightsOffset + 0x20 <= initData.length` prior to reading the length and then compute `weightsDataOffset`). This ensures that malformed data consistently reverts with the module's custom error.

Update: *Resolved in pull request #34. The team stated:*

> *Accessing the weights array length is now done after checking bounds.*

N-03 Inaccurate or Outdated Documentation

Throughout the codebase, multiple instances of inaccurate or outdated documentation were identified:

- This comment for the `ERC7579ExecutorUnexpectedOperationState` error states: "*If `allowedStates` is `bytes32(0)`, the operation is expected to not be in any state (i.e. not exist)*". However, when `allowedStates` is zero, the `require` condition in `_validateStateBitmap` becomes `0 != 0` and evaluates to `False`. Consequently, the function fails with the `ERC7579ExecutorUnexpectedOperationState` error, which implies an unexpected state as opposed to no state, thereby contradicting the comment.
- The NatSpec for weighted multisig states that "*If weights are not provided but signers are, all signers default to weight 1*". However, the implementation always decodes a third top-level element for the weights array in `_decodeMultisigWeightedInitData` and then requires equal lengths in `_setSignerWeights`. As a result, omitting the weights array can cause an "invalid init data" revert during decoding, or a length mismatch revert if an empty array is provided. As such, the documentation does not appear to be aligned with the implementation.
- Multiple instances were identified in the documentation where the `bool` type is cited as using 1 bit instead of 1 byte of storage. For example, in the `Schedule` and `ExecutionConfig` structs.
- Minor inconsistency was identified in the documentation for `scheduleAt`: "*an operation that starts its security window at `at`*" → "*an operation that starts its security window at `timepoint`*".
- A documentation error was identified in `_validateSignatures` whereby duplicates are rejected regardless of order. Ordering only optimizes the check.

Consider fixing all instances of inaccurate or incomplete documentation.

Update: *Resolved in pull request #26. The team stated:*

> *The documentation was updated.*

N-04 Redundant Return Statement

To improve code clarity, it is recommended to remove redundant return statements from functions that have named returns.

Throughout the codebase, multiple instances of redundant return statements were identified:

- The `return false;` statement in `ERC7579Multisig.sol`
- The `return hash.areValidSignaturesNow(signingSigners, signatures);` statement in `ERC7579Multisig.sol`
- The `return (signers, threshold_);` statement in `ERC7579Multisig.sol`
- The `return (signers, threshold_, weights);` statement in `ERC7579MultisigWeighted.sol`
- The `return _configs[account].delay.getFull();` statement in `ERC7579DelayedExecutor.sol`
- The `return _configs[account].expiration;` statement in `ERC7579DelayedExecutor.sol`
- The `return getSchedule(hashOperation(account, salt, mode, executionCalldata));` statement in `ERC7579DelayedExecutor.sol`
- The `return (scheduledAt, executableAt, executableAt + _schedules[operationId].expiresAfter);` statement in `ERC7579DelayedExecutor.sol`
- The `return (id, schedule_);` statement in `ERC7579DelayedExecutor.sol`
- The `return super._execute(account, salt, mode, executionCalldata);` statement in `ERC7579DelayedExecutor.sol`
- The `return initData.length > 7 ? (uint32(bytes4(initData[:4])), uint32(bytes4(initData[4:8]))) : (minSetback(), defaultExpiration());` statement in `ERC7579DelayedExecutor.sol`
- The `return returnData;` statement in `ERC7579Executor.sol`
- The `return IERC7579Execution(account).executeFromExecutor(mode, executionCalldata);` statement in `ERC7579Executor.sol`

Consider removing the redundant return statement in functions with named returns to improve code clarity and maintainability.

Update: *Resolved in pull request #35. The team stated:*

> *The team prefers explicit return statements over assigning values to named return values for readability. However, some single return values have a name, which is not common when following the OpenZeppelin Contracts guidelines that suggest single return values aren't named if they're inmediately clear.*
>
> *We decided to keep the return statements, but we're removing the name from single return values where it makes sense.*

N-05 `_removeSigners` Deletes Weight Data Before Verifying Signer Existence

The `_removeSigners` function in `ERC7579MultisigWeighted` iterates over the provided signers array and deletes their extra weight entries from storage. It does this before calling the parent implementation that validates whether each signer actually exists in the authorized set.

While this does not lead to an exploitable vulnerability because the transaction reverts entirely if any signer is not found, the order of operations is not ideal. Validation should occur before state modifications to ensure consistency and clarity. This ordering makes the code harder to reason about and could become problematic in future modifications.

Consider calling the parent implementation first to check that the signer exists before deleting their weight data.

Update: *Resolved. The team stated:*

> *We'll keep the current function ordering.*
>
> *Moving `super._removeSigners` before weight deletion would break functionality: the parent call triggers `_validateReachableThreshold`, which calculates `totalWeight()` as `getSignerCount(account) + totalExtraWeight[account]`. If weights aren't deleted first, `getSignerCount` reflects removed signers but `_totalExtraWeight` still includes their weights, creating an inconsistent state that causes incorrect threshold validation.*
>
> *The transaction reverts on invalid signers regardless, so there's no vulnerability. The order ensures both components of `totalWeight()` are synchronized before validation.*

N-06 Missing Bounds Validation for Array Data Length in Decoding Functions

The `_decodeMultisigInitData` function in `ERC7579Multisig` and the `_decodeMultisigWeightedInitData` function in `ERC7579MultisigWeighted` validate that the array offset and length field are within the bounds of `initData`. However, they do not validate that the actual array data fits within `initData`.

After reading `signersLength` from calldata, these functions set up a calldata slice using assembly without verifying that `signersDataOffset + signersLength * 0x20` does

not exceed `initData.length`. The same applies to `weightsLength` in the weighted variant. When the array length exceeds the available data, the transaction will eventually revert during iteration. However, it will do so with a generic calldata out-of-bounds error instead of the descriptive `ERC7579MultisigInvalidInitData` error that the functions already use for other validation failures.

Consider adding explicit validation that the array data fits within the provided `initData` bounds before setting up the calldata slice. For example, verify that `signersDataOffset + signersLength * 0x20 <= initData.length` for the signers array, and similarly for the weights array in the weighted variant.

Update: *Resolved in pull request #34. The team stated:*

> *Checks were added to both ERC7579Multisig and ERC7579MultisigWeighted.*

Conclusion

The audited contracts provide a clean and composable foundation for ERC-7579 smart accounts, separating execution, validation, and multisignature concerns in a way that supports broad interoperability. The implementation deliberately minimizes opinionated policy, pushing critical authorization decisions into account-side permissioning and overridable hooks. This improves flexibility, but also means that the security of any deployment depends heavily on correct integration, careful configuration, and disciplined implementation of derived modules.

Several design choices, such as delayed execution with explicit lifecycle states, configurable delay transitions, and multisig support (including weighted thresholds), enable strong security patterns when paired with appropriate operational processes (monitoring, timely execution, and controlled configuration changes). At the same time, these same features introduce integration-sensitive edge cases (e.g., lifecycle persistence across reinstall, signer list handling, and gas-growth considerations) that should be addressed explicitly in system design and documentation to avoid accidental weakening of guarantees.

Overall, the codebase was found to be safe and well-engineered, with a clean modular design that is aligned with ERC-7579/7913 patterns. However, the audit identified a critical-severity issue outside the original scope: in Enable-mode, the validation of the inner user operation signature may be bypassed, potentially allowing unauthorized operations if exploited. Beyond this, the findings are predominantly of a low severity and pertain to robustness, liveness, and developer-experience concerns: delay downgrade semantics that can bypass the intended `minSetback` floor on reinstall, missing/empty return values and documentation gaps, inability of the executor entry point to forward ETH, potential gas/memory denial-of-service vectors from unbounded return data, and edge-case validation hazards such as ERC-1271 recursion with self-signer contract accounts.

Several notes address non-security correctness and quality improvements, including redundant returns, outdated/inaccurate docs, a weighted init decoder bounds check that panics instead of reverting with a custom error, and `operationId` malleability at the ABI level that can produce different IDs for semantically equivalent operations. It is recommended to address the critical-severity finding before deployment and review the low-severity and informational issues to ensure alignment with intended security guarantees.

The OpenZeppelin Contracts team, and Ernesto Garcia in particular, are greatly appreciated for their active engagement throughout the audit and for their prompt and detailed responses to all of our technical questions.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.